

Scrum y DevOps aplicado a la mejora del modulo de Ventas en Odoo ERP

Universidad Nacional de San Agustin de Arequipa - Escuela Profesional de Ingenieria de Sistemas

Sivincha Machaca Saul Andre · Quinonez Delgado Aaron Fernando · Sencia Ale Bryan Daniel · Yauli Merma Diego Raul

Resumen – Este documento final consolida la ejecucion completa de un proyecto academico desarrollado sobre Odoo ERP. El trabajo integro Scrum, gestion documental, control de versiones, automatizacion DevOps y aseguramiento de calidad para intervenir el modulo de Ventas mediante una regla de control de descuentos. El proyecto se organizo desde Sprint 0 hasta Sprint 4, con evidencias de backlog, tablero Kanban, milestones, issues, commits, pull requests, pipelines, pruebas y publicacion en GitHub Pages. El resultado principal fue un flujo funcional que detecta descuentos superiores al limite permitido, retiene el pedido en estado `requires_review`, permite aprobacion por supervisor y habilita configuracion del limite por compania. El informe tambien documenta el cierre profesional del proyecto mediante dashboard de evidencias, burndown, defensa ejecutiva, informe final en Typst y planificacion de Jenkins como validacion complementaria.

Palabras clave – Scrum, DevOps, Odoo, ERP, GitHub Issues, GitHub Projects, GitHub Actions, Jenkins, Docker Compose, GitHub Pages, QA, trazabilidad documental.

I. Introduccion

I.A. Contexto academico del proyecto

El proyecto corresponde al trabajo final del curso Ingenieria de Procesos de Software 2026-A. Su proposito fue aplicar un ciclo completo de gestion y desarrollo sobre un producto open source real, integrando Scrum como marco de trabajo agil y practicas DevOps para versionamiento, automatizacion, pruebas, evidencias y publicacion documental. La actividad exigio que el equipo no solo produzca una modificacion tecnica, sino que demuestre el proceso seguido mediante carga documental verificable.

El producto seleccionado fue Odoo ERP, una plataforma empresarial modular cuyo codigo fuente se mantiene publicamente y permite analisis, extension e integracion de modulos funcionales [1]. La eleccion de un sistema ERP permitio trabajar con una base de codigo amplia, una arquitectura real y procesos de negocio comprensibles para un proyecto academico.

I.B. Objetivo general del trabajo

El objetivo general fue implementar y documentar una mejora funcional en Odoo ERP aplicando Scrum y practicas DevOps de forma trazable. Para cumplirlo, el equipo organizo el trabajo en cinco sprints, gestiono tareas mediante GitHub Issues y GitHub Projects, desarrollo un add-on de control de descuentos en Ventas, ejecuto pruebas, preparo evidencia de CI/CD y publico un portal de documentacion en GitHub Pages.

I.C. Producto open source seleccionado

Odoo ERP fue seleccionado por su dominio empresarial, su arquitectura modular y su aplicabilidad a procesos comerciales. El modulo principal de trabajo fue Ventas, debido a que concentra cotizaciones, pedidos, clientes, lineas de venta, confirmacion comercial y facturacion. El modulo de Compras se uso como analisis complementario para mostrar comprension del entorno ERP y de la relacion entre procesos de negocio.

I.D. Alcance tecnico y metodologico

El alcance tecnico se centro en el add-on `validacion_descuento_maximo`, que extiende el flujo de pedidos de venta para controlar descuentos superiores a un limite permitido. El alcance metodologico incluyo planificacion Scrum, seguimiento por tablero Kanban, milestones, issues, asignaciones, revisiones por sprint, QA, automatizacion con GitHub Actions, preparacion de Jenkins como validacion complementaria, publicacion en GitHub Pages y elaboracion de este informe en Typst.

I.E. Organizacion del documento

El documento se organiza como informe final y carga documental. Las primeras secciones justifican la seleccion del producto y el marco Scrum aplicado. Luego se describe la gestion en GitHub, la planificacion general, el desarrollo por sprints, la arquitectura tecnica, la implementacion funcional, DevOps, QA, trazabilidad, metricas, resultados, discusion y conclusiones. Los anexos concentran las capturas que sustentan visualmente el avance del proyecto.

II. Evidencia de seleccion y viabilidad del proyecto

II.A. Criterios de seleccion del software

La seleccion de Odoo respondio a cinco criterios: dominio conocido, licencia compatible con uso academico, complejidad suficiente, posibilidad de ejecucion local y oportunidad de extension funcional. Estos criterios permitieron trabajar con un producto no trivial, evitar una aplicacion demasiado pequena

y sostener evidencias de arquitectura, configuracion, pruebas y despliegue.

Criterio	Justificacion aplicada
Dominio	ERP/CRM con procesos empresariales reconocibles: Ventas, Compras, clientes, productos y facturacion.
Licencia	Codigo open source con licencia LGPL-3, adecuada para analisis academico y extension.
Complejidad	Base modular con multiples addons y volumen de codigo suficiente para analisis tecnico.
Ejecucion	Soporte para entorno local con Odoo y PostgreSQL mediante Docker Compose.
Extension	Permite crear addons personalizados sin modificar directamente el nucleo del producto.

II.B. Licencia y viabilidad legal

El proyecto verifico la licencia del repositorio y considero que la LGPL-3 permite estudio, ejecucion, modificacion y uso academico siempre que se respete la naturaleza del software base. Esta revision fue importante porque el trabajo no consistia en crear una aplicacion aislada, sino en intervenir un producto existente manteniendo trazabilidad sobre sus archivos y dependencias.

II.C. Complejidad del repositorio

Odoo presenta una arquitectura modular compuesta por addons, modelos, vistas, seguridad, pruebas y configuracion. Esta estructura permite evidenciar analisis de arquitectura, configuracion de entorno, pruebas automatizadas y extension funcional. El modulo `sale` y sus dependencias superan el nivel de complejidad esperado para un trabajo final, por lo que resultan adecuados para demostrar habilidades de lectura, modificacion y validacion de software existente.

II.D. Modulo principal seleccionado: Ventas

El modulo de Ventas fue seleccionado porque representa un flujo comercial central. Permite trabajar con cotizaciones, pedidos, lineas de venta, descuentos, clientes y confirmacion de operaciones. La mejora elegida se enfoco en controlar descuentos maximos, un problema funcional plausible en un proceso comercial porque descuentos no autorizados pueden afectar margenes y requerir revision de un responsable.

II.E. Modulo complementario analizado: Compras

Compras se considero como modulo complementario para ampliar el analisis ERP. Aunque la implementacion se concentro en Ventas, revisar Compras permitio entender dependencias funcionales, estructura de modulos y consistencia del producto. Esta decision ayudo a delimitar el alcance: Ventas fue el frente de implementacion; Compras fue un frente de revision y documentacion.

II.F. Capturas de repositorio, licencia y estructura del proyecto

Captura requerida: repositorio, licencia y estructura

Debe incorporarse una captura del repositorio GitHub, archivo de licencia, carpeta `addons` y modulo `sale` o add-on personalizado. Esta evidencia sustenta la seleccion y viabilidad del producto.

Figura 1: Captura requerida: repositorio, licencia y estructura

III. Marco de trabajo Scrum aplicado

III.A. Roles Scrum del equipo

El equipo asumió roles rotativos y funcionales según el avance del proyecto. Roydan lideró el Sprint 0 y participó en la configuración inicial y MVP; posteriormente dejó de participar desde Sprint 3 por reorganización del equipo. Diego lideró Sprint 1 y trabajo en documentacion, Scrum y frontend XML.

Aaron lidero Sprint 2 y asumió responsabilidades de producto, backend y QA funcional. Bryan fue Scrum Master del Sprint 3 y responsable DevOps. Saul asumió el cierre del Sprint 4 como Scrum Master, consolidando portal, evidencias, PR final e informe.

III.B. Product Backlog

El Product Backlog se formalizó mediante GitHub Issues. Cada issue representó una tarea técnica, documental, de QA, DevOps o gestión Scrum. Esta decisión permitió que el trabajo dejara de depender de acuerdos informales y quedara registrado con responsable, estado, descripción y evidencia asociada. En la vista de cierre se registraron 43 issues trazadas.

III.C. Sprint Backlog

El Sprint Backlog se construyó seleccionando issues para cada sprint. Sprint 0 se orientó a planificación y viabilidad; Sprint 1 a entorno y arquitectura; Sprint 2 al MVP y CI inicial; Sprint 3 a aprobación, seguridad y estabilización; Sprint 4 a cierre profesional, QA final, DevOps, GitHub Pages, defensa e integración.

III.D. Sprint Planning

La planificación de cada sprint definió objetivo, responsables, entregables esperados y evidencia mínima. La planificación no se limitó a una lista de tareas: se relacionó con hitos académicos y con una expectativa de cierre por sprint. Esto permitió justificar el avance ante el docente y mantener trazabilidad entre cronograma, issues y resultados.

III.E. Daily Scrum y seguimiento

El seguimiento se realizó mediante coordinación del equipo, revisión de issues, commits y tablero Kanban. Aunque no todas las reuniones quedaron registradas como actas formales, el seguimiento se evidenció en el cambio de estados, asignaciones, commits, PR e incremento de entregables por sprint. Para efectos documentales, las capturas del Project y de issues funcionan como evidencia de seguimiento.

III.F. Sprint Review

Cada Sprint Review se representó mediante revisión de entregables: plan inicial, entorno, MVP, flujo de aprobación, QA, DevOps y portal final. La revisión del Sprint 4 se concentró en verificar que el proyecto pueda defenderse: páginas públicas, dashboard, burndown, defensa ejecutiva, evidencias y PR de integración.

III.G. Sprint Retrospective

La retrospectiva se refleja en ajustes sucesivos del proceso. El equipo pasó de una configuración inicial y documentación dispersa a un backlog más formal, luego a pruebas automatizadas, después a artifacts y finalmente a un portal público con dashboard y documento final. La reorganización del equipo después de Sprint 2 también obligó a redistribuir responsabilidades y cerrar el trabajo con cuatro integrantes activos.

III.H. Definition of Done

La Definition of Done del proyecto se definió como una combinación de criterios: issue cerrada, responsable identificado, evidencia documental, cambio versionado, prueba o validación cuando aplica, enlace en GitHub Pages y trazabilidad hacia el sprint correspondiente. En Sprint 4 se agregó como criterio adicional que los entregables fueran defendibles ante el docente.

III.I. Evidencias de aplicación Scrum

Captura requerida: Scrum aplicado

Debe incluir Product Backlog, tablero Kanban, hitos, issues por sprint, responsables y estados de cierre. Estas capturas demuestran la aplicación de Scrum y no solo su descripción textual.

Figura 2: Captura requerida: Scrum aplicado

IV. Gestión del proyecto en GitHub

IV.A. Organización del repositorio

El repositorio se organizó sobre la rama base 19.0, con ramas de feature para cambios específicos y pull requests para integración. La documentación académica se reorganizó hacia docs/, mientras que el portal público se mantuvo en archivos Markdown de raíz y layout Jekyll. El cierre del Sprint 4 se trabajó en feature/sprint4-github-pages, rama usada para mejorar GitHub Pages, resolver conflictos contra 19.0, agregar el informe Typst y preparar la entrega final.

IV.B. Uso de GitHub Issues

GitHub Issues se utilizó como unidad principal de trabajo. Las issues permitieron registrar tareas de producto, QA, Scrum, DevOps, documentación, Pages e integración final. Para Sprint 4 se destacaron issues como portal público, validación funcional, QA final, documentación Scrum, DevOps final, entregables profesionales, insumos finales y PR de cierre.

IV.C. Uso de GitHub Projects como tablero Kanban

GitHub Projects funcionó como tablero Kanban para visualizar estados y seguimiento. Su uso permitió representar el flujo de trabajo y defender el avance frente al docente. Las columnas y estados del Project permiten demostrar movimiento de tareas desde backlog hasta cierre.

IV.D. Hitos por sprint e hito

Los hitos agruparon trabajo por sprint e hito. Esta estructura fue relevante porque el curso evaluó entregas parciales y cierre final. Relacionar issues con hitos facilita demostrar que el avance no fue aislado, sino organizado según el cronograma maestro.

IV.E. Labels y clasificación de tareas

Las tareas se clasificaron por frente: Scrum, producto, QA, DevOps, documentación, Pages e integración. Esta clasificación permitió distribuir responsabilidades individuales y evitar que todos aparezcan como responsables de la misma tarea. También permitió separar trabajo técnico de trabajo documental.

IV.F. Asignación de responsables

La asignación se realizó a una persona por issue cuando fue posible. Esto ayudó a sostener responsabilidad individual. En la vista documental se consolidaron asignaciones aceptadas por integrante: Saul 8, Aaron 9, Bryan 9, Diego 9 y Roydan 3. Roydan no figura desde Sprint 3 por reorganización del equipo.

IV.G. Capturas del tablero, issues, hitos y backlog

Captura requerida: gestión en GitHub

Debe incorporarse el Project Kanban, la lista de hitos, filtros de issues por sprint y asignaciones individuales.

Figura 3: Captura requerida: gestión en GitHub

V. Planificación general y cronograma

V.A. Cronograma maestro

El cronograma maestro organizó el trabajo desde el 1 de mayo hasta el 13 de julio. Se definió un Sprint 0 de planificación y cuatro sprints de desarrollo, estabilización y cierre. Este cronograma permitió vincular el avance con los hitos del curso y justificar la evolución gradual del proyecto.

V.B. Distribución de hitos

El Hito 1 se relacionó con selección, planificación y viabilidad. El Hito 2 cubrió implementación inicial, pruebas y CI. El Hito 3 concentró estabilización, cierre, evidencias, QA, DevOps y publicación documental. Esta distribución facilitó mostrar progreso incremental y no una entrega concentrada al final.

V.C. Fechas de cada sprint

Sprint	Fechas	Resultado temporal documentado
Sprint 0	01 mayo - 13 mayo	Concluido con seleccion, alcance y cronograma.
Sprint 1	14 mayo - 28 mayo	Concluido con entorno, arquitectura y backlog inicial.
Sprint 2	29 mayo - 10 junio	Concluido con MVP y CI inicial.
Sprint 3	11 junio - 26 junio	Concluido con flujo de aprobacion y artifact.
Sprint 4	27 junio - 13 julio	Concluido con cierre documental y PR final.

V.D. Objetivos por sprint

Sprint 0 busco seleccionar Odoo, definir alcance, equipo, cronograma y viabilidad. Sprint 1 preparo entorno, analisis Ventas/Compras y organizo backlog. Sprint 2 implemento el MVP de validacion de descuento maximo e inicio CI. Sprint 3 evoluciono el flujo con supervisor, aprobacion, limite configurable y artifacts. Sprint 4 cerro el proyecto con GitHub Pages, QA final, DevOps, documentacion, defensa e integracion.

V.E. Entregables esperados por sprint

Los entregables incluyeron plan inicial, cronograma, entorno Docker, analisis tecnico, addon personalizado, pruebas, workflow, artifacts, dashboard de evidencias, burndown, defensa ejecutiva, informe Typst y PR final. Esta combinacion demuestra entregables tecnicos y documentales.

V.F. Capturas del cronograma y planificacion

Captura requerida: cronograma y planificacion

Debe agregarse captura del cronograma maestro y de la pagina de sprints del portal.

Figura 4: Captura requerida: cronograma y planificacion

VI. Desarrollo documentado por sprints

VI.A. Sprint 0: seleccion, planificacion y viabilidad

VI.A.A. Objetivo del sprint

Seleccionar el producto open source, validar su viabilidad legal y tecnica, definir el alcance inicial, organizar el equipo y establecer el cronograma maestro.

VI.A.B. Actividades realizadas

Se reviso el repositorio de Odoo, se identifico el modulo de Ventas como frente principal, se verifico licencia, se definio el equipo, se preparo el cronograma y se inicio la estructura documental del proyecto.

VI.A.C. Roles y responsables

Roydan Apaza lidero el Sprint 0. Saul apoyo cronograma y backlog inicial; Aaron reviso viabilidad tecnica; Bryan reviso arquitectura y complejidad; Diego gestiono documentacion inicial.

VI.A.D. Issues asociadas

La vista de cierre documenta cinco tareas cerradas para Sprint 0, orientadas a seleccion, planificacion, cronograma, viabilidad y gestion inicial.

VI.A.E. Evidencias documentales y capturas

Captura requerida: Sprint 0

Capturas de repositorio, licencia, cronograma inicial, issues y tablero.

Figura 5: Captura requerida: Sprint 0

VI.A.F. Resultado del sprint

El sprint cerro con producto seleccionado, alcance definido, equipo organizado, cronograma base y viabilidad tecnica documentada.

VI.B. Sprint 1: configuracion del entorno y analisis arquitectonico

VI.B.A. Objetivo del sprint

Preparar el entorno de trabajo, analizar arquitectura de Ventas y Compras, organizar el backlog tecnico e iniciar practicas DevOps.

VI.B.B. Actividades realizadas

Se trabajo en configuracion de Odoo y PostgreSQL con Docker Compose, analisis de dependencias del modulo sale, revision de vistas XML, organizacion de ramas y formalizacion de issues.

VI.B.C. Roles y responsables

Diego Yauli fue lider del Sprint 1. Roydan participo en configuracion de entorno; Aaron analizo arquitectura; Bryan apoyo DevOps y evidencias; Saul gestiono backlog e issues.

VI.B.D. Issues asociadas

El sprint se documento con siete issues cerradas y ocho asignaciones aceptadas, vinculadas a entorno, arquitectura, backlog y soporte DevOps.

VI.B.E. Evidencias documentales y capturas

Captura requerida: Sprint 1

Capturas de Docker Compose, arquitectura de Ventas/Compras, tablero e informe de Sprint 1.

Figura 6: Captura requerida: Sprint 1

VI.B.F. Resultado del sprint

El sprint cerro con base tecnica y organizativa suficiente para iniciar implementacion funcional.

VI.C. Sprint 2: implementacion inicial e integracion CI/CD

VI.C.A. Objetivo del sprint

Implementar la primera version funcional del addon de validacion de descuento maximo e iniciar integracion continua.

VI.C.B. Actividades realizadas

Se desarrollo el MVP del addon validacion_descuento_maximo, se agregaron pruebas funcionales, se integro la interfaz inicial en Ventas y se configuro GitHub Actions para validaciones automatizadas.

VI.C.C. Roles y responsables

Aaron Quinonez lidero el Sprint 2. Roydan trabajo en backend MVP; Diego integro vistas XML; Bryan preparo GitHub Actions y validacion Docker; Saul consolido QA y evidencias.

VI.C.D. Issues asociadas

El sprint registro cinco issues cerradas y cinco asignaciones aceptadas, relacionadas con backend, frontend, DevOps, QA y gestion.

VI.C.E. Evidencias documentales y capturas

Captura requerida: Sprint 2

Capturas del MVP, pruebas, workflow de GitHub Actions, issue list y tablero.

Figura 7: Captura requerida: Sprint 2

VI.C.F. Resultado del sprint

El sprint entrego la primera version funcional del control de descuento maximo y una base inicial de CI.

VI.D. Sprint 3: mejoras, refactorizacion y estabilizacion

VI.D.A. Objetivo del sprint

Evolucionar el control de descuentos hacia un flujo de aprobacion parametrizable, estable y validado por pruebas.

VI.D.B. Actividades realizadas

Se incorporo el estado `requires_review`, el grupo Supervisor de Descuentos, la accion de aprobacion, el limite configurable por compania, pruebas parametrizadas y artifact de resultados.

VI.D.C. Roles y responsables

Bryan Sencia fue Scrum Master del Sprint 3 y trabajo en pipeline/artifacts. Aaron trabajo en backend, aprobacion y parametro. Diego trabajo en frontend XML y documentacion. Saul trabajo en QA del flujo.

VI.D.D. Issues asociadas

El sprint registro doce issues cerradas y doce asignaciones aceptadas, concentradas en backend, frontend, QA, DevOps y documentacion.

VI.D.E. Evidencias documentales y capturas

Captura requerida: Sprint 3

Capturas del estado `'requires_review'`, aprobacion, limite configurable, Actions y artifact.

Figura 8: Captura requerida: Sprint 3

VI.D.F. Resultado del sprint

El desarrollo funcional principal quedo completo, con flujo de aprobacion y validacion automatizada.

VI.E. Sprint 4: cierre profesional, QA final y documentacion

VI.E.A. Objetivo del sprint

Transformar el trabajo tecnico en una entrega final verificable, navegable y evaluable, integrando evidencias Scrum, QA, DevOps, documentacion y presentacion publica mediante GitHub Pages.

VI.E.B. Actividades realizadas

Se construyo un portal profesional con paleta Odoo, dashboard de evidencias, pagina de sprints, burndown, defensa ejecutiva, reorganizacion documental, PR final, informe Typst y planificacion de Jenkins como validacion complementaria.

VI.E.C. Roles y responsables

Saul Sivincha asumio el rol de Scrum Master del cierre. Aaron trabajo en validacion funcional y QA final. Bryan asumio DevOps final, Docker Compose, Actions, Jenkinsfile y badges. Diego trabajo en documentacion Scrum, entregables, roadmap, guia y presentacion. Roydan no participo desde Sprint 3.

VI.E.D. Issues asociadas

Las issues finales incluyeron portal GitHub Pages, validacion funcional Ventas/Compras, QA final, trazabilidad Scrum, DevOps final, entregables profesionales, insumos finales e integracion final. La vista de cierre identifica #56, #65, #66, #67, #68, #69, #70 y #33 como referencias principales.

VI.E.E. Evidencias documentales y capturas

Captura requerida: Sprint 4

Capturas del portal, dashboard de evidencias, burndown, PR final, Actions, Jenkins y PDF Typst.

Figura 9: Captura requerida: Sprint 4

VI.E.F. Resultado del sprint

El sprint cerro el Hito 3 con una entrega documental y tecnica defendible: portal publico, evidencias, dashboard, burndown, QA, DevOps e informe final.

VII. Arquitectura tecnica del producto

VII.A. Arquitectura general de Odoo

Odoo se organiza como plataforma modular. Sus addons encapsulan modelos, vistas, seguridad, datos, pruebas y configuracion. Esta arquitectura permite extender comportamientos

sin alterar directamente el nucleo del sistema, lo que resulta adecuado para un proyecto academico de mejora funcional [1].

VII.B. Modulo de Ventas

El modulo de Ventas administra cotizaciones, pedidos, clientes, productos, lineas de venta, descuentos y confirmacion comercial. Por ello fue el modulo principal para implementar una regla de control de descuentos maximos.

VII.C. Modulo de Compras

Compras se reviso como complemento del analisis ERP. Su inclusion documental permite mostrar que el equipo no se limito a leer un unico archivo, sino que comprendio el contexto empresarial de Odoo y sus procesos relacionados.

VII.D. Addon personalizado

El addon `validacion_descuento_maximo` encapsula la logica de validacion, estados, vistas, configuracion, seguridad y pruebas asociadas al control de descuentos. Su uso respeta el patron de extension de Odoo.

VII.E. Estructura de archivos modificados

Archivo o carpeta	Funcion
<code>models/sale_order.py</code>	Extension del pedido de venta y control del flujo de revision.
<code>models/res_company.py</code>	Parametro empresarial del limite de descuento.
<code>models/res_config_settings.py</code>	Exposicion del limite configurable en ajustes.
<code>views/sale_order_views.xml</code>	Boton y elementos visibles del flujo de aprobacion.
<code>views/res_config_settings_views.xml</code>	Vista de configuracion del limite.
<code>security/discount_security.xml</code>	Grupo Supervisor de Descuentos.
<code>tests/</code>	Pruebas automatizadas del comportamiento funcional.

VII.F. Flujo funcional implementado

El flujo revisa descuentos en lineas de pedido. Si el descuento se mantiene dentro del limite, el pedido continua su confirmacion normal. Si supera el limite, el pedido pasa a `requires_review`. Un supervisor autorizado puede aprobar el descuento y permitir que el pedido continue el flujo comercial.

VII.G. Capturas de estructura, codigo y vistas

Captura requerida: arquitectura tecnica

Capturas de estructura del addon, archivos modificados, vistas XML y modelos principales.

Figura 10: Captura requerida: arquitectura tecnica

VIII. Implementacion funcional

VIII.A. Validacion de descuento maximo

La validacion compara el porcentaje de descuento de las lineas de venta con el limite configurado. Esta regla evita que descuentos superiores al umbral avancen sin revision comercial.

VIII.B. Estado `requires_review`

El estado `requires_review` representa pedidos retenidos por superar el limite de descuento. Esta decision evita depender solo de errores bloqueantes y permite un flujo de aprobacion trazable.

VIII.C. Grupo Supervisor de Descuentos

El grupo Supervisor de Descuentos restringe la accion de aprobacion a usuarios autorizados. Este control separa al vendedor comun del responsable que puede aprobar excepciones comerciales.

VIII.D. Boton de aprobacion

El boton de aprobacion aparece en la interfaz del pedido de venta cuando corresponde. Su objetivo es permitir que el supervisor confirme la excepcion y continúe el proceso.

VIII.E. Limite configurable

El limite configurable permite ajustar el porcentaje maximo desde configuracion de compania. Esto evita dejar la regla fija en codigo y facilita adaptar el comportamiento a distintos escenarios.

VIII.F. Vistas XML

Las vistas XML integran visualmente el flujo en Odoo. Incluyen elementos en el formulario del pedido y configuracion del limite, permitiendo que el usuario interactúe con la mejora desde la interfaz.

VIII.G. Evidencias visuales del flujo en Odoo

Captura requerida: flujo funcional en Odoo

Capturas de pedido con descuento, estado `requires_review`, boton de aprobacion, usuario supervisor y configuracion del limite.

Figura 11: Captura requerida: flujo funcional en Odoo

IX. DevOps y automatizacion

IX.A. Docker Compose

Docker Compose se utilizo para reproducir el entorno Odoo + PostgreSQL. Esta decision redujo dependencias manuales y permitio levantar servicios de manera consistente durante desarrollo, QA y validaciones [2].

IX.B. GitHub Actions

GitHub Actions se utilizo como herramienta de integracion continua. Los workflows validaron configuracion, ejecucion de pruebas y generacion de evidencia mediante logs y artifacts [3].

IX.C. Ejecucion automatizada de pruebas

Las pruebas automatizadas verificaron escenarios del limite de descuento, estado de revision, aprobacion y configuracion. Su ejecucion en CI permitio detectar regresiones antes de consolidar cambios en la rama base.

IX.D. Artifacts y logs

Los artifacts y logs permiten conservar evidencia de ejecucion. En un contexto academico, estos elementos son importantes porque muestran que la validacion no fue solo declarada, sino ejecutada y registrada.

IX.E. Jenkins como validacion complementaria

Jenkins se plantea como herramienta DevOps complementaria. Su rol esperado es ejecutar stages de checkout, validacion de Docker Compose, pruebas del modulo y publicacion de evidencia. No sustituye GitHub Actions; demuestra conocimiento de una alternativa CI/CD ampliamente usada [4].

IX.F. GitHub Pages como publicacion documental

GitHub Pages se uso como publicacion documental del proyecto. El portal concentra sprints, evidencias, burndown, defensa, arquitectura, verificacion tecnica y enlaces de cierre. Esta publicacion facilita la evaluacion porque el docente puede navegar el proyecto sin revisar archivos aislados [5].

IX.G. Capturas de pipelines, artifacts y publicacion

Captura requerida: DevOps y publicacion

Capturas de GitHub Actions, artifacts, Jenkins, Docker Compose y GitHub Pages.

Figura 12: Captura requerida: DevOps y publicacion

X. Aseguramiento de calidad

X.A. Estrategia de QA

La estrategia de QA combino pruebas funcionales, pruebas automatizadas y verificacion documental. El foco fue comprobar que el flujo de descuentos respondiera correctamente ante descuentos normales, descuentos superiores al limite, usuarios no supervisores, usuarios supervisores y cambios de configuracion.

X.B. Casos de prueba

Caso	Resultado esperado	Evidencia
Descuento permitido	El pedido continua el flujo normal.	Prueba funcional.
Descuento excedido	El pedido pasa a <code>requires_review</code> .	Prueba funcional.
No supervisor	No puede aprobar excepciones.	Prueba de permisos.
Supervisor	Puede aprobar y continuar.	Prueba de rol.
Limite configurable	La regla cambia segun configuracion.	Prueba parametrizada.

X.C. Pruebas funcionales

Las pruebas funcionales validan el comportamiento desde la perspectiva del usuario y del proceso comercial. Su objetivo es confirmar que la regla de negocio sea visible y defendible en Odoo.

X.D. Pruebas de roles y permisos

Las pruebas de roles comprueban que solo usuarios autorizados puedan aprobar descuentos retenidos. Esta validacion es central porque la mejora tiene sentido si existe separacion de responsabilidades.

X.E. Pruebas de configuracion

Las pruebas de configuracion verifican que el limite configurable cambie el comportamiento del sistema. Esto confirma que la solucion no depende de un porcentaje fijo y puede adaptarse.

X.F. Resultados de QA

La evidencia documentada registra pruebas cerradas y sin pendientes para la defensa. Las ejecuciones automatizadas y logs de CI respaldan la estabilidad del flujo implementado.

X.G. Capturas de pruebas y resultados

Captura requerida: QA

Capturas de pruebas automatizadas, logs de GitHub Actions, artifact y validacion funcional en Odoo.

Figura 13: Captura requerida: QA

XI. Trazabilidad del proyecto

XI.A. Relacion entre sprints, issues y entregables

Cada sprint se vinculo con issues y entregables concretos. Esto permite rastrear el camino desde planificacion hasta producto final: Sprint 0 define, Sprint 1 prepara, Sprint 2 implementa, Sprint 3 estabiliza y Sprint 4 documenta e integra.

XI.B. Relacion entre responsables y tareas

La asignacion individual permitio mostrar contribucion por integrante. Saul aparece asociado al cierre, portal y PR; Aaron a producto y QA; Bryan a DevOps; Diego a documentacion, Scrum y frontend; Roydan a configuracion inicial y MVP antes de su salida.

XI.C. Relacion entre commits y funcionalidades

El historial evidencia commits funcionales y de CI: configuracion Docker (3763f4d), MVP de descuento (0d8f0b), pruebas QA (758796c), visualizacion XML (d775046), supervisor (14576d), limite configurable (08d2ee), correcciones de CI y

reorganizacion documental (8a652d). Estos commits permiten conectar codigo y entregables.

XI.D. Relacion entre pruebas y criterios de aceptacion

Los criterios de aceptacion se vinculan con pruebas: descuento permitido, descuento excedido, aprobacion por supervisor, restriccion a no supervisor y limite configurable. Esta relacion es clave para demostrar calidad y cierre.

XI.E. Matriz de trazabilidad completa

Sprint	Entregable	Evidencia	Estado
0	Seleccion y viabilidad	Cronograma, licencia, repositorio	Cerrado
1	Entorno y arquitectura	Docker, informe, backlog	Cerrado
2	MVP y CI inicial	Addon, pruebas, Actions	Cerrado
3	Aprobacion y parametrizacion	Supervisor, artifacts, tests	Cerrado
4	Cierre documental	Pages, burndown, PR, informe	Cerrado

XII. Metricas y seguimiento Scrum

XII.A. Issues creadas por sprint

La vista de evidencias registra 43 issues trazadas. Por sprint, la distribucion usada para cierre fue: Sprint 0 con 5, Sprint 1 con 7, Sprint 2 con 5, Sprint 3 con 12 y Sprint 4 con 14.

XII.B. Issues cerradas por sprint

La vista de cierre documenta 43 issues cerradas y 0 abiertas para defensa. Esto representa un cierre documental completo del backlog previsto para evaluacion.

XII.C. Issues aceptadas por integrante

Las asignaciones aceptadas se distribuyeron en Saul 8, Aaron 9, Bryan 9, Diego 9 y Roydan 3. Roydan solo aparece hasta Sprint 2 debido a la reorganizacion del equipo.

XII.D. Burndown del proyecto

El burndown muestra reduccion del trabajo pendiente hasta llegar a cero en Sprint 4. Su funcion es evidenciar control del avance, no solo mostrar una grafica estetica.

XII.E. Avance acumulado

El avance acumulado paso de planificacion inicial a cierre profesional. En la representacion documental, Sprint 0 inicia el proyecto, Sprint 2 alcanza un MVP, Sprint 3 completa la funcionalidad principal y Sprint 4 consolida evidencias y publicacion.

XII.F. Dashboard de evidencias

El dashboard de evidencias separa la informacion por sprint, mostrando objetivo, lider, issues creadas, asignaciones, roles y enlaces de verificacion. Esto permite una lectura rapida para defensa.

XII.G. Capturas de graficos y metricas

Captura requerida: metricas Scrum

Capturas del dashboard de evidencias, burndown, graficos por sprint e issues cerradas.

Figura 14: Captura requerida: metricas Scrum

XIII. Resultados del proyecto

XIII.A. Resultados funcionales

Se obtuvo un flujo funcional de control de descuentos en Ventas: validacion de limite, estado de revision, grupo supervisor, aprobacion y configuracion. Esto representa un incremento verificable sobre Odoo.

XIII.B. Resultados metodologicos Scrum

El proyecto evidencia uso de Scrum mediante roles, sprints, backlog, issues, tablero, milestones, responsables, reviews documentadas y cierre por sprint.

XIII.C. Resultados DevOps

Se documento Docker Compose, GitHub Actions, pruebas automatizadas, logs, artifacts, GitHub Pages y planificacion de Jenkins. Estos elementos muestran practicas DevOps aplicadas a un repositorio real.

XIII.D. Resultados documentales

El portal GitHub Pages, el dashboard de evidencias, el burndown, la defensa ejecutiva, la reorganizacion docs/ y el informe Typst forman la carga documental final.

XIII.E. Cumplimiento de hitos

El Hito 1 se cubrio con seleccion y planificacion; el Hito 2 con MVP y CI inicial; el Hito 3 con estabilizacion, QA, DevOps, portal, PR e informe final.

XIII.F. Estado final del producto

El producto queda documentado como incremento funcional y academico, con evidencias suficientes para evaluacion. Las tareas pendientes se concentran en completar capturas reales, implementar o evidenciar Jenkins y cerrar integracion final si el PR aun no se ha fusionado.

XIV. Discusion

XIV.A. Decisiones tecnicas relevantes

La decision mas importante fue pasar de una validacion bloqueante simple a un flujo de revision con estado requires_review. Esto permite trazabilidad de excepciones y aprobacion por supervisor.

XIV.B. Problemas encontrados

El equipo enfrente reorganizacion de integrantes, integracion de ramas, conflictos con 19.0, necesidad de ordenar documentacion dispersa y estabilizar CI para ejecutar pruebas en un entorno controlado.

XIV.C. Riesgos gestionados

Se gestionaron riesgos de falta de evidencia, cambios no trazables, issues sin responsable, conflictos de merge, rutas rotas en GitHub Pages y validaciones CI incompletas.

XIV.D. Limitaciones del proyecto

El proyecto se concentro en el modulo de Ventas. Compras fue analizado, pero no intervenido funcionalmente. Jenkins se considera complementario y debe mostrarse como implementado solo si se ejecuta y documenta con capturas.

XIV.E. Lecciones aprendidas

El equipo aprendio que Scrum requiere evidencias y no solo reuniones declaradas; que DevOps necesita scripts reproducibles; que GitHub Pages puede funcionar como carga documental; y que integrar cambios en un repositorio grande exige resolver conflictos con criterio.

XV. Conclusiones

XV.A. Conclusiones sobre el producto

La mejora implementada aporta control comercial al flujo de Ventas de Odoo, permitiendo detectar y aprobar descuentos superiores al limite configurado.

XV.B. Conclusiones sobre Scrum

Scrum permitio organizar el avance por objetivos, roles, issues y entregables. La trazabilidad por sprints facilita defender el proceso ante evaluacion academica.

XV.C. Conclusiones sobre DevOps

Docker Compose, GitHub Actions, artifacts y GitHub Pages fortalecieron la reproducibilidad y evidencia del proyecto.

Jenkins puede ampliar esa evidencia como herramienta CI/CD complementaria.

XV.D. Conclusiones sobre QA

QA permitio validar que la regla de descuento, permisos y configuracion funcionen de manera coherente. La relacion entre casos de prueba y criterios de aceptacion es esencial para sostener calidad.

XV.E. Trabajo futuro

Como trabajo futuro se recomienda completar Jenkins con pipeline ejecutable, agregar capturas finales en anexos, ampliar pruebas de borde, integrar reportes mas detallados y evaluar una extension funcional al modulo de Compras.

XVI. Referencias

Bibliografia

- [1] Odoo S.A., «Odoo 19.0 Source Code». [En línea]. Disponible en: <https://github.com/odoo/odoo>
- [2] Docker Inc., «Docker Compose Documentation». [En línea]. Disponible en: <https://docs.docker.com/compose/>
- [3] GitHub, «GitHub Actions Documentation». [En línea]. Disponible en: <https://docs.github.com/actions>
- [4] Jenkins Project, «Jenkins User Documentation». [En línea]. Disponible en: <https://www.jenkins.io/doc/>
- [5] GitHub, «GitHub Pages Documentation». [En línea]. Disponible en: <https://docs.github.com/pages>

XVII. Anexos

XVII.A. Anexo A. Capturas completas del Product Backlog

Carga documental

Insertar capturas completas del Product Backlog.

XVII.B. Anexo B. Capturas del tablero Kanban

Carga documental

Insertar capturas del tablero GitHub Projects.

XVII.C. Anexo C. Capturas de milestones

Carga documental

Insertar capturas de milestones por sprint e hito.

XVII.D. Anexo D. Capturas de issues por sprint

Carga documental

Insertar capturas filtradas de issues Sprint 0 a Sprint 4.

XVII.E. Anexo E. Capturas de pull requests

Carga documental

Insertar capturas del PR final y PRs relevantes.

XVII.F. Anexo F. Capturas de commits relevantes

Carga documental

Insertar capturas del historial de commits funcionales, QA y DevOps.

XVII.G. Anexo G. Capturas de GitHub Actions

Carga documental

Insertar capturas de workflows, logs y artifacts.

XVII.H. Anexo H. Capturas de Jenkins

Carga documental

Insertar capturas si Jenkins queda implementado y ejecutado.

XVII.I. Anexo I. Capturas de Docker Compose

Carga documental

Insertar capturas del entorno levantado y servicios.

XVII.J. Anexo J. Capturas de GitHub Pages

Carga documental

Insertar capturas de portal, evidencias, burndown y defensa.

XVII.K. Anexo K. Capturas del flujo funcional en Odoo

Carga documental

Insertar capturas de descuento, revision, aprobacion y configuracion.

XVII.L. Anexo L. Matriz completa de trazabilidad

Carga documental

Insertar matriz final sprint-issue-responsible-deliverable-evidence.